

Project: Claim Wise – Automated Claims Processing System for Health Insurance

Introduction

This document provides the Low-Level Design (LLD) for **Claim Wise**, an automated health insurance claims processing system. Claim Wise streamlines claim submission, automates eligibility checks, manages approvals, and delivers analytics to improve fraud detection and operational efficiency. The system is organized into key modules covering policyholder profiles, claim submission, eligibility rules, approval workflows, and fraud analytics.

Module 1: Policyholder Profile & Coverage

This module handles the core data and operations related to policyholders, their coverage types, and associated agents. It is designed with clean separation across layers to ensure maintainability, testability, and onboarding clarity.

Backend (Clean Architecture)

1. Domain Layer

This layer defines the **core business entities** and **interfaces** that represent the heart of the insurance domain.

Entities

- **Policyholder:** Represents the insured individual. Includes personal details, contact info, and coverage linkage.
- **Policy Type:** Defines coverage categories (e.g., health, accident). Used for eligibility and premium logic.
- **Agent:** Represents the intermediary managing policyholder onboarding and servicing.

Interfaces

- **IPolicyholder, IPolicyType, IAgent:** Abstract contracts for domain operations (e.g., validations, rules).

2. Infrastructure Layer

This layer connects domain logic to data persistence using repositories.

Repositories

- **Policyholder Repository**
 - **PolicyType Repository**
 - **Agent Repository**
-

3. Application Layer

This layer orchestrates business logic, DTO mapping, and service coordination.

DTOs

- **PolicyholderDto**
- **PolicyTypeDto**
- **AgentDto**

Interfaces

- **IPolicyholder Service**
- **IPolicyType Service**
- **IAgent Service**

Services

- **Policyholder Service**
 - **PolicyType Service**
 - **Agent Service**
-

4. API Layer

This layer exposes RESTful endpoints for frontend consumption.

Controllers

- **Policyholder Controller**

- **PolicyType Controller**
 - **Agent Controller**
-

CODE – (Policyholder Controller)

```
using AutoMapper;
using ClaimWise.Application.DTOs;
using ClaimWise.Domain.Entities;
using ClaimWise.Domain.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Threading.Tasks;
namespace ClaimWise.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PolicyholderController : ControllerBase
    {
        private readonly IPolicyholderRepository _policyholderRepository;
        private readonly IDependentRepository _dependentRepository;
        private readonly IAgentRepository _agentRepository;
        private readonly IPolicyTypeRepository _policyTypeRepository;
        private readonly IMapper _mapper;
        public PolicyholderController(
```

```

    IPolicyholderRepository policyholderRepository,
    IDependentRepository dependentRepository,
    IAgentRepository agentRepository,
    IPolicyTypeRepository policyTypeRepository,
    IMapper mapper)
{
    _policyholderRepository = policyholderRepository;
    _dependentRepository = dependentRepository;
    _agentRepository = agentRepository;
    _policyTypeRepository = policyTypeRepository;
    _mapper = mapper;
}

// GET: api/Policyholder
[HttpGet]
[Authorize(Roles = "Admin")]
[ProducesResponseType(typeof(IEnumerable<PolicyholderDto>),
StatusCodes.Status200OK)]
public async Task<IActionResult> GetAll()
{
    var policyholders = await _policyholderRepository.GetAllAsync();
    var policyholderDtos = _mapper.Map<IEnumerable<PolicyholderDto>>(policyholders);
    return Ok(policyholderDtos);
}

// GET: api/Policyholder/self
[HttpGet("self")]
[Authorize(Roles = "Policyholder")]
[ProducesResponseType(typeof(PolicyholderDto), StatusCodes.Status200OK)]

```

```

[ProducesResponseType(StatusCodes.Status404NotFound)]
public async Task<IActionResult> GetLoggedInPolicyholder()
{
    var userName = User.Identity?.Name;
    if (string.IsNullOrEmpty(userName))
        return Unauthorized("User name not found in token.");
    var all = await _policyholderRepository.GetAllAsync();
    var match = all.LastOrDefault(p => p.Name.ToLower() == userName.ToLower());
    if (match == null)
        return NotFound("Policyholder record not found.");
    var dto = _mapper.Map<PolicyholderDto>(match);
    return Ok(dto);
}

// POST: api/Policyholder
[HttpPost]
[Authorize(Roles = "Policyholder")]
[ProducesResponseType(typeof(PolicyholderDto), StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
public async Task<IActionResult> Create([FromBody] PolicyholderDto policyholderDto)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    // Set name from authenticated user
    var role =
User.FindFirst("http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

    var name = User.Identity?.Name ?? "Unknown";
    if (role == "Policyholder")

```

```
{
    policyholderDto.Name = name;
}

// Validate AgentID against existing agents
var validAgents = await _agentRepository.GetAllAsync();
var allowedAgentIds = validAgents.Select(a => a.AgentID).ToList();
if (!allowedAgentIds.Contains(policyholderDto.AgentID))
{
    return BadRequest($"Invalid Agent ID. Please choose one of the allowed agents:
{string.Join(", ", allowedAgentIds)}");
}

// Fetch PolicyType to get PolicyDurationYears and CoverageDetails
var policyType = await
_policyTypeRepository.GetByIdAsync(policyholderDto.PolicyTypeID);
if (policyType == null)
{
    return BadRequest($"Invalid PolicyTypeID: {policyholderDto.PolicyTypeID}");
}

// Auto-fill CoverageDetails if not provided
if (string.IsNullOrEmpty(policyholderDto.CoverageDetails))
{
    policyholderDto.CoverageDetails = policyType.Description;
}

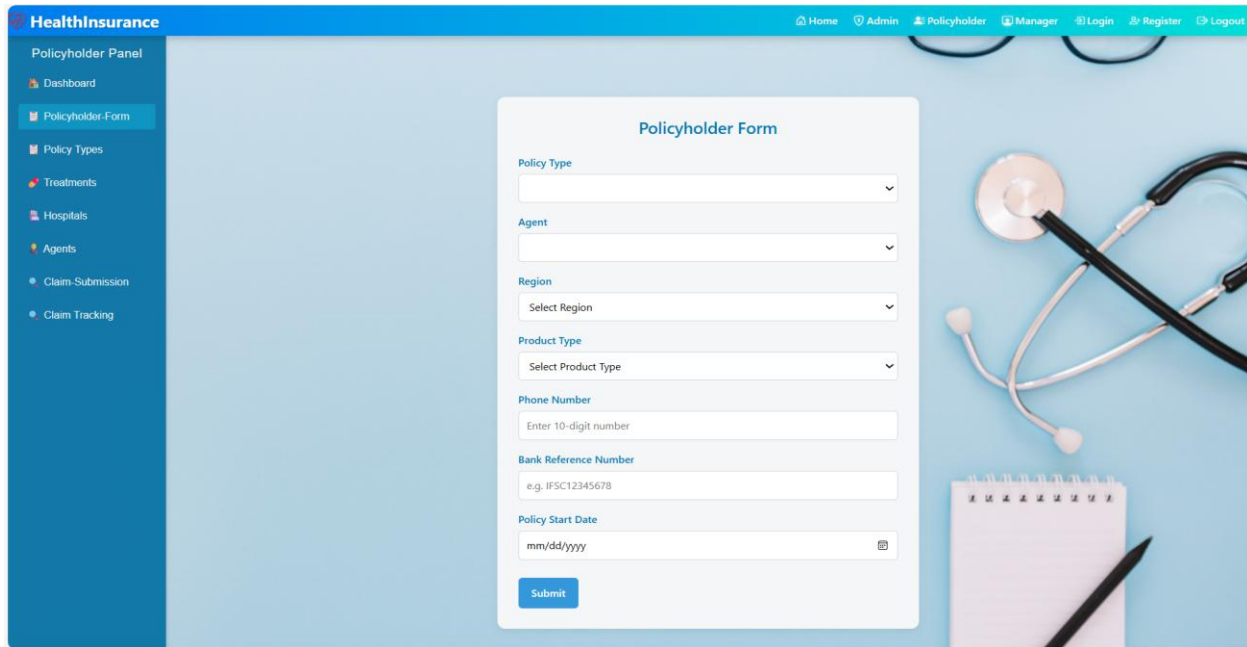
// Calculate PolicyEndDate using PolicyStartDate + PolicyDurationYears
var policyEndDate =
policyholderDto.PolicyStartDate.AddYears(policyType.PolicyDurationYears);

// Map DTO to entity
var policyholder = new Policyholder
```

```
{
    Name = policyholderDto.Name,
    PolicyTypeID = policyholderDto.PolicyTypeID,
    CoverageDetails = policyholderDto.CoverageDetails,
    AgentID = policyholderDto.AgentID,
    Region = policyholderDto.Region,
    ProductType = policyholderDto.ProductType,
    PhoneNumber = policyholderDto.PhoneNumber,
    BankReferenceNumber = policyholderDto.BankReferenceNumber,
    PolicyStartDate = policyholderDto.PolicyStartDate,
    PolicyEndDate = policyholderDto.PolicyEndDate
};
// Save to database
await _policyholderRepository.AddAsync(policyholder);
// return created DTO
var createdDto = _mapper.Map<PolicyholderDto>(policyholder);
return CreatedAtAction(nameof(GetAll), new { id = policyholder.PolicyholderID },
createdDto);
}
```

Frontend (Angular + Bootstrap)

This layer provides the user interface for managing policyholder profiles and coverage.



The screenshot displays a web application titled "HealthInsurance". The top navigation bar includes links for Home, Admin, Policyholder, Manager, Login, Register, and Logout. A left sidebar menu lists various options: Policyholder Panel, Dashboard, Policyholder-Form (highlighted), Policy Types, Treatments, Hospitals, Agents, Claim-Submission, and Claim Tracking. The main content area features a "Policyholder Form" with the following fields: Policy Type (dropdown), Agent (dropdown), Region (dropdown with "Select Region" text), Product Type (dropdown with "Select Product Type" text), Phone Number (text input with "Enter 10-digit number" placeholder), Bank Reference Number (text input with "e.g. IFSC12345678" placeholder), and Policy Start Date (text input with "mm/dd/yyyy" placeholder and a calendar icon). A blue "Submit" button is located at the bottom of the form. The background of the form area shows a stethoscope and a notepad.

Components

- **policyholder-form**: Form to add policyholder details.
- **policy-type**: Displays available coverage types.
- **agents**: Lists agents and their assigned policyholders.

Services

- **policyholder**
- **policytype**
- **agent**
- **dependent** (for managing dependents under a policyholder)

CODE - (policyholder.service.ts)

```
import { Injectable } from '@angular/core';
```



```

import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from '../auth.service';
@Injectable({ providedIn: 'root' })
export class PolicyholderService {
  private baseUrl = 'http://localhost:5030/api';
  constructor(private http: HttpClient, private auth: AuthService) {}
  getAgents(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseUrl}/Agent`, {
      headers: this.auth.getAuthHeaders()
    });
  }
  getPolicyTypes(): Observable<any[]> {
    return this.http.get<any[]>(`${this.baseUrl}/PolicyType`, {
      headers: this.auth.getAuthHeaders()
    });
  }
  create(dto: any): Observable<any> {
    return this.http.post<any>(`${this.baseUrl}/Policyholder`, dto, {
      headers: this.auth.getAuthHeaders()
    });
  }
  addDependents(policyholderID: number, dependents: any): Observable<any> {
    return this.http.post<any>(
      `${this.baseUrl}/Policyholder/${policyholderID}/dependents`,
      dependents,
      { headers: this.auth.getAuthHeaders() }
    );
  }
}

```

);}}

Module 2: Claim Submission & Document Upload Module

In this module, I worked on implementing the claim submission and document upload process. The main purpose of this module is to allow policyholders to submit their claims, upload supporting documents, and enable admin users to manage and track those claims efficiently.

Backend (Clean Architecture)

1. Domain Layer

This layer defines the core business entities and interfaces. It represents the structure of the data and the key business logic without depending on any external frameworks.

Entities

- **Claim:** Represents the claim request details submitted by the policyholder.
- **Hospital:** Stores hospital-related information linked to the claim.
- **Treatment:** Captures treatment details like type, cost, and duration.
- **Policyholder:** Represents the insured person submitting the claim.
- **DocumentAccessLog:** Maintains logs for document uploads and access history.
- **ClaimActionLog:** Records claim-related actions such as submission, approval, or rejection.

Interfaces

Each entity will have a corresponding interface to define the contract for data access or operations.

- **Claim**
 - **Hospital**
 - **Treatment**
 - **Policyholder**
 - **DocumentAccessLog**
 - **ClaimActionLog**
-

2. Infrastructure Layer

This layer handles data persistence and repository management. It connects the domain entities to the database.

Repositories

- **ClaimRepository**
 - **HospitalRepository**
 - **TreatmentRepository**
 - **PolicyholderRepository**
 - **DocumentAccessLogRepository**
 - **ClaimActionLogRepository**
-

3. Application Layer

The application layer handles business logic and service operations. It connects the domain and infrastructure layers, processes user inputs, and coordinates workflows like claim submission or document upload.

DTOs (Data Transfer Objects)

DTOs are used to transfer data between layers without exposing the internal structure of entities.

- **Claim**
- **Hospital**
- **Treatment**
- **Policyholder**
- **DocumentAccessLog**
- **ClaimActionLog**

Interfaces

These define service-level contracts for handling business operations.

- **Claim**
- **Hospital**
- **Treatment**
- **Policyholder**
- **DocumentAccessLog**
- **ClaimActionLog**

Services

Services handle the main business logic using repositories.

- **ClaimService**
 - **HospitalService**
 - **TreatmentService**
 - **PolicyholderService**
 - **DocumentAccessLogService**
 - **ClaimActionLogService**
 - **WhatsAppService** (used for notification or status updates)
-

4. API Layer

This layer exposes RESTful endpoints for external communication. It interacts with the frontend to process requests and send responses.

Controllers

Controllers handle HTTP requests and return appropriate responses.

- **ClaimController**
 - **HospitalController**
 - **TreatmentController**
 - **PolicyholderController**
 - **WhatsAppController**
-

CODE (Claim Controller)

```
using System;

using System.Collections.Generic;

using System.IO;

using System.Linq;

using System.Threading.Tasks;

using ClaimWise.Application.DTOs;
```

```
using ClaimWise.Application.Helpers;
using ClaimWise.Application.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration;
namespace ClaimWise.API.Controllers
{
    [Authorize]
    [ApiController]
    [Route("api/[controller]")]
    public class ClaimController : ControllerBase
    {
        private readonly IClaimService _claimService;
        private readonly IAuditLogService _auditLogService;
        private readonly IClaimActionLogService _claimActionLogService;
        private readonly IConfiguration _configuration;
        private readonly IClaimRepository _claimRepository;
        private readonly IEligibilityCheckRepository _eligibilityCheckRepository;
        public ClaimController(
            IClaimService claimService,
            IAuditLogService auditLogService,
            IClaimActionLogService claimActionLogService,
            IConfiguration configuration,
            IClaimRepository claimRepository,
            IEligibilityCheckRepository eligibilityCheckRepository)
        {
```

```

        _claimService = claimService;
        _auditLogService = auditLogService;
        _claimActionLogService = claimActionLogService;
        _configuration = configuration;
        _claimRepository = claimRepository;
        _eligibilityCheckRepository = eligibilityCheckRepository;
    }

    [Authorize(Roles = "Admin")]
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var claims = await _claimService.GetAllClaimsAsync();
        return Ok(claims);
    }

    [Authorize(Roles = "Policyholder")]
    [HttpPost("submit")]
    [Consumes("multipart/form-data")]
    public async Task<IActionResult> SubmitClaimWithFiles([FromForm] CreateClaimDto
dto)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var existingClaim = await
_claimService.GetClaimByPolicyholderIdAsync(dto.PolicyholderID);

        if (existingClaim != null)
            return BadRequest("You have already submitted a claim. Only one claim is allowed
per policyholder.");

        if (dto.Documents == null || dto.Documents.Count < 1 || dto.Documents.Count > 2)

```

```

        return BadRequest("You must upload 1 or 2 documents only.");
var fileNames = dto.Documents.Select(f => f.FileName.ToLower()).ToList();
if (fileNames.Distinct().Count() != fileNames.Count)
    return BadRequest("Uploaded documents must have different filenames.");
foreach (var file in dto.Documents)
{
    if (file.Length > 5 * 1024 * 1024)
        return BadRequest($"File {file.FileName} exceeds 5 MB limit.");
    var allowedExtensions = new[] { ".pdf", ".jpg", ".jpeg", ".png" };
    var extension = Path.GetExtension(file.FileName).ToLower();
    if (!allowedExtensions.Contains(extension))
        return BadRequest($"File {file.FileName} has unsupported format. Allowed: PDF,
JPG, JPEG, PNG.");
    var fileName = Path.GetFileNameWithoutExtension(file.FileName);
    if (fileName.Contains("scan", StringComparison.OrdinalIgnoreCase) ||
        fileName.Contains("copy", StringComparison.OrdinalIgnoreCase))
        return BadRequest($"File {file.FileName} appears to be a fraud document (contains
'scan' or 'copy').");
}
var claimId = await _claimService.SubmitClaimAsync(dto);
await _claimActionLogService.LogAsync(claimId, User.Identity?.Name ?? "Unknown",
"Claim Submitted");
return Ok($"Claim {claimId} submitted successfully with validated documents.");
}
[Authorize(Roles = "Admin")]
[HttpPut("{id}/Review")]
public async Task<IActionResult> UpdateStatus(int id, [FromBody] UpdateClaimStatusDto
dto)

```

```

{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    dto.UpdatedBy = User.Identity?.Name ?? "Unknown";
    dto.Status = "UnderReview";

    var claim = await _claimService.GetClaimByIdAsync(id);
    if (claim == null)
        return NotFound($"Claim with ID {id} not found.");

    if (claim.Status == "Approved")
        return BadRequest("Cannot update status of an already approved claim.");

    var updated = await _claimService.UpdateClaimStatusAsync(id, dto);
    if (!updated)
        return BadRequest("Status update failed. Invalid transition or claim already approved.");

    await _claimActionLogService.LogAsync(id, dto.UpdatedBy, "Claim moved to UnderReview");

    return Ok("Claim status updated to UnderReview.");
}

[Authorize(Roles = "Admin")]
[HttpPut("{claimId}/status")]
public async Task<IActionResult> ApproveClaim(int claimId)
{
    var approvedBy = User.Identity?.Name ?? "Unknown";
    var claim = await _claimService.GetClaimByIdAsync(claimId);
    if (claim == null)
        return NotFound($"Claim with ID {claimId} not found.");

    if (claim.Status == "Approved")
        return BadRequest("Claim is already approved.");
}

```



```

        if (claim.Status != "UnderReview")
            return BadRequest("Claim must be under review before it can be approved.");

        var latestCheck = await
_eligibilityCheckRepository.GetLatestByClaimIdAsync(claimId);

        if (latestCheck == null)
            return BadRequest("Eligibility check not found. Run eligibility check before
approval.");

        if (latestCheck.Result == "Not Eligible")
        {
            await _claimRepository.UpdateStatusAsync(claimId, "Rejected");
            await _claimActionLogService.LogAsync(claimId, approvedBy, "Claim Rejected");
            var rejectedClaim = await _claimRepository.GetByIdAsync(claimId);
            await _claimService.NotifyPolicyholderStatusChangeAsync(rejectedClaim);
            return Ok($"Claim {claimId} rejected based on eligibility check.");
        }

        await _claimRepository.UpdateStatusAsync(claimId, "Approved");
        await _claimActionLogService.LogAsync(claimId, approvedBy, "Claim Approved");
        var approvedClaim = await _claimRepository.GetByIdAsync(claimId);
        await _claimService.NotifyPolicyholderStatusChangeAsync(approvedClaim);
        return Ok($"Claim {claimId} approved successfully.");
    }

    [Authorize(Roles = "Admin,Policyholder")]
    [HttpGet("download/{claimId}")]
    public async Task<IActionResult> DownloadClaimDocument(int claimId)
    {
        var claim = await _claimRepository.GetByIdAsync(claimId);

        if (claim == null || string.IsNullOrEmpty(claim.Documents))
            return NotFound("Claim or document not found.");
    }

```

```

        var userId = User.Identity?.Name ?? "Unknown";

        var userRole =
            User.FindFirst("http://schemas.microsoft.com/ws/2008/06/identity/claims/role")?.Value;

        if (!string.Equals(userRole, "Admin", StringComparison.OrdinalIgnoreCase) &&
            claim.PolicyholderID.ToString() != userId)
        {
            return Unauthorized("You are not authorized to download this document.");
        }

        var filePath = claim.Documents.Split(';').First();
        var fileName = Path.GetFileName(filePath);
        if (!System.IO.File.Exists(filePath))
            return NotFound("Encrypted file not found.");

        var encryptedBytes = await System.IO.File.ReadAllBytesAsync(filePath);
        var encryptionKey = _configuration["Encryption:AESKey"];
        var decryptedBytes = DocumentEncryptionHelper.Decrypt(encryptedBytes,
            encryptionKey)

        var ip = HttpContext.Connection.RemoteIpAddress?.ToString();
        var device = Request.Headers["User-Agent"].ToString();
        var action = $"Downloaded {fileName} from {device} ({ip})";
        await _auditLogService.LogAccessAsync(claim.ClaimID, userId, fileName, action);
        return File(decryptedBytes, "application/octet-stream", fileName);
    }

    [Authorize(Roles = "Admin")]
    [HttpGet("{claimId}/audit")]
    public async Task<ActionResult> GetAuditLogs(int claimId, [FromQuery] int
        pageNumber = 1, [FromQuery] int pageSize = 10)
    {
        var logs = await _auditLogService.GetLogsAsync(claimId, pageNumber, pageSize);
    }

```

```

        return Ok(logs);
    }

    [Authorize(Roles = "Admin,Policyholder")]
    [HttpGet("{claimId}/actions")]
    public async Task<IActionResult> GetActionLogs(int claimId, [FromQuery] int
pageNumber = 1, [FromQuery] int pageSize = 10)
    {
        var logs = await _claimActionLogService.GetLogsAsync(claimId, pageNumber,
pageSize);
        return Ok(logs);
    }

    [Authorize(Roles = "Admin")]
    [HttpGet("status-counts")]
    public async Task<IActionResult> GetClaimStatusCounts()
    {
        var counts = await _claimService.GetClaimStatusCountsAsync();
        return Ok(counts);
    }

    [Authorize(Roles = "Admin")]
    [HttpGet("Filtered claims")]
    public async Task<IActionResult> GetClaimsByStatus([FromQuery] string status)
    {
        var claims = await _claimService.GetAllClaimsAsync();
        var filtered = claims.Where(c => c.Status.Equals(status,
StringComparison.OrdinalIgnoreCase));
        return Ok(filtered);
    }
}

```

}

Frontend (Angular + Bootstrap)

This module provides a user interface and backend integration for submitting and tracking insurance claims, securely uploading supporting documents, and ensuring validated communication with the claims processing system.

The screenshot displays the 'HealthInsurance' application interface. On the left is a blue sidebar with a 'Policyholder Panel' containing links to Dashboard, Policyholder-Form, Policy Types, Treatments, Hospitals, Agents, Claim-Submission (highlighted), and Claim Tracking. The main content area features a 'Submit Your Claim' form with the following fields: Policyholder ID, Hospital ID, Treatment ID, and Treatment Details (a text area). Below these is a document upload section labeled 'Upload Documents (PDF/JPG/PNG)' with a 'Choose Files' button and a 'No file chosen' status. A blue 'Submit Claim' button is at the bottom of the form. The top navigation bar includes links for Home, Admin, Policyholder, Manager, Login, Register, and Logout. The browser's address bar shows 'localhost:4200/home'.

Components

Each component represents a specific feature or page of the application.

- **Treatments:** Displays available treatment details and related claim information.
- **Hospitals:** Shows a list of hospitals and allows selection for claim linkage.
- **Claims:** Handles the form for submitting and uploading claim-related documents.
- **Claims-Tracking:** Displays the claim status, actions taken, and document access logs.

Services

Angular services handle API communication between frontend and backend.

- **Treatment Service:** Fetches treatment-related data from the backend.
- **Hospital Service:** Retrieves hospital details and links them to claims.
- **Claim Service:** Manages claim submission and document upload functions.

- **Claims-Tracking Service:** Tracks claim status and activity logs.

CODE (Claim Service)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from '../auth.service';
@Injectable({ providedIn: 'root' })
export class ClaimService {
  private baseUrl = 'http://localhost:5030/api';
  constructor(private http: HttpClient, private auth: AuthService) {}
  submitClaim(formData: FormData): Observable<any> {
    const headers = this.auth.getAuthHeaders();
    return this.http.post(`${this.baseUrl}/Claim/submit`, formData, { headers });
  }
  getHospitalIDs(): Observable<number[]> {
    return this.http.get<number[]>(`${this.baseUrl}/Hospital`);
  }
  getTreatmentIDs(): Observable<{ id: number; name: string }[]> {
    return this.http.get<{ id: number; name: string }[]>(`${this.baseUrl}/TreatmentHospital`);
  }
}
```

Module 3: Eligibility & Rule Engine Module

The main goal of this module is to validate claim eligibility based on predefined rules such as waiting periods, coverage type, and treatment category. It ensures that only eligible claims move forward for processing and settlement.

1. Domain Layer

This layer defines the core business entities and interfaces for managing claim details and eligibility checks. It focuses on representing the essential data and business logic independent of other layers.

Entities

- **Claim:** Represents the claim details submitted by the policyholder, including claim amount, treatment details, and policy info.
- **EligibilityCheck:** Holds all the information required to determine whether a particular claim is eligible based on the defined business rules.

Interfaces

- **Claim**
 - **EligibilityCheck**
-

2. Infrastructure Layer

This layer handles data persistence and repository management. It connects the domain entities to the database.

Repositories

- **ClaimRepository**
 - **EligibilityCheckRepository**
-

3. Application Layer

This layer handles the business logic and rule evaluation process. It coordinates between the domain and infrastructure layers to execute eligibility checks, validate claim conditions, and trigger notifications.

DTOs (Data Transfer Objects)

- **EligibilityCheck:** Represents the input data required to perform the eligibility validation.
- **EligibilityResult:** Contains the output of the rule engine after evaluating a claim's eligibility (e.g., Eligible, Not Eligible, or Pending).

Interfaces

- **EligibilityCheck:** Defines the operations for performing and managing eligibility checks.
 - **WhatsApp:** Used for sending notifications or alerts to policyholders regarding their claim eligibility status.
-

4. API Layer

The API layer exposes endpoints for frontend interaction. It allows the frontend to submit claim details, trigger eligibility checks, and fetch eligibility results through RESTful APIs.

Controllers

- **EligibilityCheckController:** Handles API requests related to checking claim eligibility, running rule validations, and returning eligibility results to the frontend.
-

CODE (EligibilityCheckController)

```
using System.Collections.Generic;
using System.Threading.Tasks;
using AutoMapper;
using ClaimWise.Application.DTOs;
using ClaimWise.Application.Interfaces;
using ClaimWise.Application.Services;
```

```
using ClaimWise.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
namespace ClaimWise.API.Controllers
{
    [Authorize]
    [ApiController]
    [Route("api/[controller]")]
    public class EligibilityCheckController : ControllerBase
    {
        private readonly IEligibilityCheckService _eligibilityCheckService;
        private readonly IMapper _mapper;
        public EligibilityCheckController(
            IEligibilityCheckService eligibilityCheckService,
            IMapper mapper)
        {
            _eligibilityCheckService = eligibilityCheckService;
            _mapper = mapper;
        }
        [Authorize(Roles = "Admin,Agent")]
        [HttpPost("check")]
        public async Task<IActionResult> RunCheck([FromBody] RunEligibilityRequestDto dto)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);
            var checkedBy = User.Identity?.Name ?? "Unknown";
            // Enforce claim status check before eligibility
```



```

var claim = await _eligibilityCheckService.GetClaimByIdAsync(dto.ClaimID);

if (claim == null)

    return NotFound($"Claim with ID {dto.ClaimID} not found.");

if (claim.Status != "UnderReview")

    return BadRequest("Eligibility check can only be run when claim is under review.");

var result = await _eligibilityCheckService.RunEligibilityCheckAsync(dto.ClaimID,
checkedBy);

return Ok(result);

}

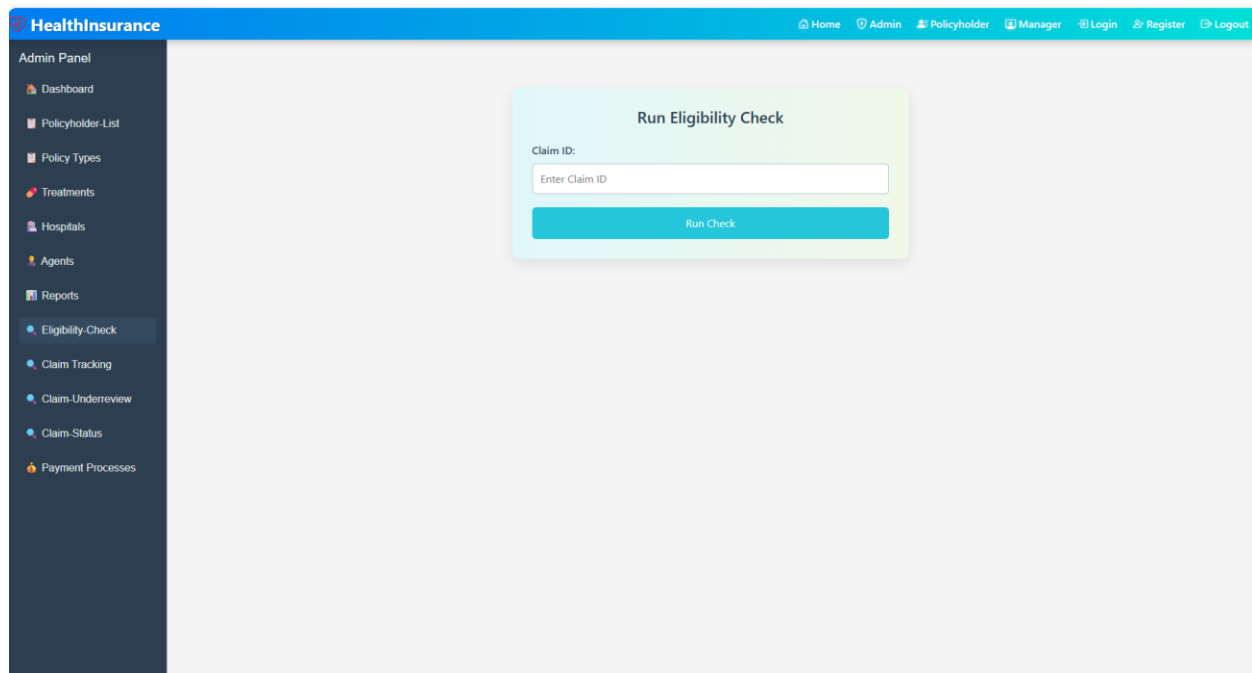
}

}

```

Frontend (Angular + Bootstrap)

The frontend provides an interactive user interface for policyholders and admins to view claim status, perform eligibility checks, and track claim progress visually.



Components

- **Claim-Underview:** Displays an overview of claim details and helps users verify claim information before running eligibility checks.
 - **Eligibility-Check:** Allows users to perform the eligibility check based on claim details and displays the rule evaluation result.
 - **Claim-Status:** Shows the current eligibility and processing status of the claim in a simple view.
-

Services

- **Claim-Underview Service:** Fetches and displays claim information from the backend before eligibility validation.
 - **Eligibility-Check Service:** Sends claim details to the backend for eligibility validation and retrieves rule engine results.
 - **Claim-Status Service:** Tracks and displays the eligibility decision or claim progress after validation.
-

CODE (Eligibility-Check Service)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { AuthService } from '../auth.service';

export interface EligibilityResult {
  claimID: number;
  result: string;
  rulesApplied: string[];
}

@Injectable({ providedIn: 'root' })
export class EligibilityCheckService {
```

```
private baseUrl = 'http://localhost:5030/api/EligibilityCheck';
constructor(private http: HttpClient, private auth: AuthService) {}
runEligibilityCheck(claimID: number): Observable<EligibilityResult> {
  const headers = this.auth.getAuthHeaders();
  const dto = { claimID };
  return this.http.post<EligibilityResult>(`${this.baseUrl}/check`, dto, { headers }).pipe(
    catchError(err => {
      const message = typeof err.error === 'string' ? err.error : 'Eligibility check failed.';
      return throwError(() => new Error(message));
    })
  );
}
```

Module 4: Approval Workflow & Payout Module

In this module, I focused on implementing the Approval Workflow and Payout process. The main objective of this module is to handle claim approval, manage multiple levels of authorization, and process the final payout to the policyholder once the claim is approved.

This module ensures that only verified and eligible claims are approved and that the payout is calculated and disbursed accurately. The workflow integrates business rules, approval logic, and notification services for seamless communication.

1. Domain Layer

The domain layer defines the core entities and interfaces related to claim approvals and payouts. It captures the business structure and key entities involved in the approval process.

Entities

- **Claim:** Represents the claim details that require approval before payout. It stores information like claim amount, policyholder details, and approval status.
- **Payout:** Represents the payout information generated after the claim is approved. It includes payout amount, payment status, and transaction details.

Interfaces

These interfaces define the contracts that connect the entities to their corresponding repositories and services.

- **Claim**
 - **Payout**
-

2. Infrastructure Layer

This layer deals with database operations and data persistence. It manages how claim and payout data are stored, retrieved, and updated in the database.

Repositories

- **ClaimRepository**
 - **PayoutRepository**
-

3. Application Layer

This layer contains the core business logic for approval workflows and payout processing. It bridges the communication between the domain and infrastructure layers and ensures that all approval rules and payment procedures are executed properly.

DTOs (Data Transfer Objects)

- **Claim:** Used to transfer claim data between the backend layers during approval.
- **Payout:** Used to transfer payout-related data between layers after claim approval.
- **PayoutResult:** Contains the final output or result after a payout is processed (e.g., Success, Failed, Pending).

Services

- **Claim Service:** Manages the claim approval workflow, including approval stages and validation.
 - **Payout Service:** Handles payout calculation, transaction creation, and payment updates.
 - **WhatsApp Service:** Sends notifications or alerts to policyholders regarding approval or payout status.
-

4. API Layer

The API layer provides RESTful endpoints to interact with the frontend. It handles incoming requests for approval and payout operations and sends back the results.

Controllers

- **PayoutController:** Manages API requests related to payout processing, approval confirmation,

CODE (PayoutController)

```
using System.Collections.Generic;
```

```
using System.Threading.Tasks;
```

```
using AutoMapper;
```

```
using ClaimWise.Application.DTOs;
```

```
using ClaimWise.Application.Interfaces;
```

```
using ClaimWise.Domain.Entities;
```

```
using ClaimWise.Domain.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
namespace ClaimWise.API.Controllers
{
    [Authorize]
    [ApiController]
    [Route("api/[controller]")]
    public class PayoutController : ControllerBase
    {
        private readonly IPayoutRepository _payoutRepository;
        private readonly IPayoutService _payoutService;
        private readonly IClaimActionLogService _logService;
        private readonly IClaimService _claimService;
        private readonly IPolicyholderRepository _policyholderRepository;
        private readonly IMapper _mapper;
        public PayoutController(
            IPayoutRepository payoutRepository,
            IPayoutService payoutService,
            IClaimActionLogService logService,
            IClaimService claimService,
            IPolicyholderRepository policyholderRepository,
            IMapper mapper)
        {
            _payoutRepository = payoutRepository;
            _payoutService = payoutService;
            _logService = logService;
        }
    }
}
```

```

        _claimService = claimService;
        _policyholderRepository = policyholderRepository;
        _mapper = mapper;
    }

    // Admin-only: Calculate payout for a claim
    [Authorize(Roles = "Admin")]
    [HttpPost("calculate")]
    public async Task<IActionResult> Calculate([FromBody] CalculatePayoutRequestDto
dto)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);

        var claim = await _claimService.GetClaimByIdAsync(dto.ClaimID);

        if (claim == null)
            return NotFound($"Claim with ID {dto.ClaimID} not found.");

        if (claim.Status == "Rejected")
            return BadRequest("Payout cannot be calculated for a rejected claim.");

        var performedBy = User.Identity?.Name ?? "Unknown";

        var result = await _payoutService.CalculatePayoutAsync(dto.ClaimID,
performedBy);

        return Ok(result);
    }

    // Admin or Manager: Get all payouts
    [Authorize(Roles = "Admin,Manager")]
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var payouts = await _payoutRepository.GetAllAsync();
    }

```

```

var payoutDtos = _mapper.Map<IEnumerable<PayoutDto>>(payouts);
foreach (var dto in payoutDtos)
{
    if (dto.ApprovedByLevel1 == "Pratheek")
        dto.ApprovedByLevel1 = "Pratheek (Manager)";
    if (dto.ApprovedByLevel2 == "Lance")
        dto.ApprovedByLevel2 = "Lance (Admin)";
}
return Ok(payoutDtos);
}

// Manager-only: Approve payout at Level 1
[Authorize(Roles = "Manager")]
[HttpPut("{payoutId}/Manager_Approve-level1")]
public async Task<IActionResult> ApproveLevel1(int payoutId)
{
    var username = User.Identity?.Name;
    if (username != "Pratheek")
        return StatusCode(403, new { message = "Only Pratheek is allowed to approve Level 1 payouts." });
    var payout = await _payoutRepository.GetByIdAsync(payoutId);
    if (payout == null)
        return NotFound(new { message = $"Payout with ID {payoutId} not found." });
    var claim = await _claimService.GetClaimByIdAsync(payout.ClaimID);
    if (claim == null)
        return NotFound(new { message = $"Claim with ID {payout.ClaimID} not found." });
    if (claim.Status == "Rejected")

```



```

        return BadRequest(new { message = "Rejected claims cannot be approved for
payout." });

        if (!string.IsNullOrEmpty(payout.ApprovedByLevel1) &&
payout.Level1ApprovalDate.HasValue)

            return BadRequest(new { message = "Payout already approved at Level 1." });

        await _payoutService.ApproveLevel1Async(payoutId, username);

        await _payoutService.NotifyPolicyholderLevel1ApprovalAsync(payout);

        return Ok(new { message = $"Payment {payoutId} approved at Level 1 by Manager."
});
    }

    // Admin-only: Approve payout at Level 2
    [Authorize(Roles = "Admin")]
    [HttpPut("{payoutId}/Admin_Approve-level2")]
    public async Task<IActionResult> ApproveLevel2(int payoutId)
    {
        var payout = await _payoutRepository.GetByIdAsync(payoutId);

        if (payout == null)

            return NotFound($"Payout with ID {payoutId} not found.");

        var claim = await _claimService.GetClaimByIdAsync(payout.ClaimID);

        if (claim == null)

            return NotFound($"Claim with ID {payout.ClaimID} not found.");

        if (claim.Status == "Rejected")

            return BadRequest("Rejected claims cannot be approved for payout.");

        if (string.IsNullOrEmpty(payout.ApprovedByLevel1) ||
!payout.Level1ApprovalDate.HasValue)

            return BadRequest("Level 1 approval must be completed before Level 2.");

        if (!string.IsNullOrEmpty(payout.ApprovedByLevel2) &&
payout.Level2ApprovalDate.HasValue)

            return BadRequest("Payout already approved at Level 2.");

```

```

        var approvedBy = User.Identity?.Name ?? "Unknown";
        await _payoutService.ApproveLevel2Async(payoutId, approvedBy);
        return Ok(new { message = $"Payment {payoutId} approved at Level 1 by Manager."
    });
}

// Admin-only: Mark payout as transferred
[Authorize(Roles = "Admin")]
[HttpPut("{payoutId}/transfer")]
public async Task<IActionResult> MarkAsTransferred(int payoutId)
{
    var payout = await _payoutRepository.GetByIdAsync(payoutId);
    if (payout == null)
        return NotFound($"Payout with ID {payoutId} not found.");
    var claim = await _claimService.GetClaimByIdAsync(payout.ClaimID);
    if (claim == null)
        return NotFound($"Claim with ID {payout.ClaimID} not found.");
    if (claim.Status == "Rejected")
        return BadRequest("Rejected claims cannot be transferred.");
    var policyholder = await
        _policyholderRepository.GetByIdAsync(claim.PolicyholderID);
    if (policyholder == null)
        return NotFound($"Policyholder with ID {claim.PolicyholderID} not found.");
    if (string.IsNullOrEmpty(policyholder.BankReferenceNumber))
        return BadRequest("Bank reference number is missing for this policyholder.");
    var updatedBy = User.Identity?.Name ?? "Unknown";
    await _payoutService.MarkAsTransferredAsync(payoutId,
        policyholder.BankReferenceNumber, updatedBy);
    await _payoutService.NotifyPolicyholderPayoutAsync(payout);
}

```

```

        return Ok(new { message = $"Payment {payoutId} marked as transferred and
notification sent." });
    }

    // Admin or Manager: Fetch claim logs
    [Authorize(Roles = "Admin,Manager")]
    [HttpGet("claim/{claimId}/logs")]
    public async Task<IActionResult> GetClaimLogs(int claimId, int page = 1, int size =
10)
    {
        var claim = await _claimService.GetClaimByIdAsync(claimId);
        if (claim == null)
            return NotFound($"Claim with ID {claimId} not found.");
        if (claim.Status == "Rejected")
            return BadRequest("Logs are not available for rejected claims.");
        var logs = await _logService.GetLogsAsync(claimId, page, size);
        return Ok(logs);
    }
}
}

```

Frontend (Angular + Bootstrap)

The frontend layer is designed to provide an interactive interface for admin users to review, approve, and process claim payouts. It also displays payment statuses and approval results in real-time.

Components

- **Payment-Process:** Manages the user interface for initiating and monitoring the payout process. It displays payout details and allows admins to trigger payment actions.
 - **Approval-1:** Represents the first approval stage in the workflow where the admin reviews claim details before authorizing payout.
-

Services

- **Payment-Process Service:** Connects the frontend to backend payout APIs to fetch payout details and trigger payment processing.
- **Approval-1 Service:** Communicates with backend APIs to fetch claims pending approval and updates approval decisions.

CODE (Payment-Process Service)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { AuthService } from '../auth.service';
import { Observable } from 'rxjs';
@Injectable({ providedIn: 'root' })
export class PaymentProcessesService {
  private baseUrl = 'http://localhost:5030/api/Payout';
  constructor(private http: HttpClient, private auth: AuthService) {}
  calculatePayout(claimID: number): Observable<any> {
    return this.http.post(`${this.baseUrl}/calculate`, { claimID }, {
      headers: this.auth.getAuthHeaders()
    });
  }
}
```

```
}  
approveLevel2(payoutId: number): Observable<any> {  
    return this.http.put(`${this.baseUrl}/${payoutId}/Admin_Approve-level2`, {}, {  
        headers: this.auth.getAuthHeaders()  
    });  
}  
  
markAsTransferred(payoutId: number): Observable<any> {  
    return this.http.put(`${this.baseUrl}/${payoutId}/transfer`, {}, {  
        headers: this.auth.getAuthHeaders()  
    });  
}  
  
// New: Get all payouts  
getAllPayouts(): Observable<any[]> {  
    return this.http.get<any[]>(`${this.baseUrl}`, {  
        headers: this.auth.getAuthHeaders()  
    });  
}  
}
```

Module 5: Claims Report Module

In this module, I focused on implementing the Claims Report functionality, which is used to generate detailed reports based on claim data. The purpose of this module is to provide administrators and stakeholders with insights into claim activities, approval rates, and payout summaries.

This module allows users to generate customized reports based on selected parameters such as date, claim type, hospital, or status. It ensures that data is well-organized and easy to analyze for better decision-making.

1. Domain Layer

The domain layer defines the core entities and interfaces required to represent claim and reporting data. It acts as the foundation for all the data structures used across the module.

Entities

- **Claim:** Represents the claim data that will be used for report generation. It includes claim ID, policyholder information, hospital details, and claim status.
- **ClaimsReport:** Represents the final report details generated from the claim data, including summaries, totals, and categorized information.

Interfaces

These interfaces define the contracts for how the entities interact with repositories and services in other layers.

- **Claim**
 - **ClaimsReport**
-

2. Infrastructure Layer

This layer is responsible for database operations and data access related to claim reporting. It manages data fetching, filtering, and aggregation from the claim records to prepare for report generation.

Repositories

- **ClaimRepository:** Used to retrieve claim data from the database that serves as input for report generation.
- **ClaimsReportRepository:** Used to store and manage generated reports for future reference or export.

3. Application Layer

The application layer handles the business logic and report generation process. It connects the domain and infrastructure layers, processes claim data and converts it into structured reports based on input filters.

DTOs (Data Transfer Objects)

- **ClaimsReport:** Represents the report details that will be displayed or exported in the frontend.
- **GenerateReportRequest:** Contains parameters or filters (like date range, hospital, claim type) provided by the user to generate specific reports.

Interfaces

- **Report:** Defines methods and operations for generating, retrieving, and formatting claim reports.

Services

- **Report Service:** Handles the main business logic of generating reports by combining claim data, applying filters, and creating formatted outputs.
-

4. API Layer

The API layer exposes RESTful endpoints for the frontend to interact with. It allows users to request report generation and retrieve the generated results.

Controllers

- **ClaimsReportController:** Manages API endpoints related to generating reports, fetching report data, and providing downloadable or viewable results to the frontend.

CODE (ClaimsReportController)

```
using System;  
  
using System.Collections.Generic;  
  
using System.Text;  
  
using System.Threading.Tasks;  
  
using AutoMapper;  
  
using ClaimWise.Application.DTOs;  
  
using ClaimWise.Application.Interfaces;
```

```
using ClaimWise.Domain.Entities;
using ClaimWise.Domain.Interfaces;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
namespace ClaimWise.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    //[Authorize(Roles = "Admin")] // Restrict entire controller to Admins
    public class ClaimsReportController : ControllerBase
    {
        private readonly IClaimsReportRepository _claimsReportRepository;
        private readonly IReportService _reportService;
        private readonly IMapper _mapper;
        public ClaimsReportController(
            IClaimsReportRepository claimsReportRepository,
            IReportService reportService,
            IMapper mapper)
        {
            _claimsReportRepository = claimsReportRepository;
            _reportService = reportService;
            _mapper = mapper;
        }
        // GET: All reports
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
```



```

        var reports = await _claimsReportRepository.GetAllAsync();
        var reportDtos = _mapper.Map<IEnumerable<ClaimsReportDto>>(reports);
        return Ok(reportDtos);
    }

    // POST: Auto-generate report for a specific claim
    [HttpPost("Auto-generateReport")]
    public async Task<ActionResult> GenerateFraudReport([FromBody]
GenerateReportRequestDto dto)
    {
        if (dto == null || dto.ClaimId <= 0)
            return BadRequest("Invalid claim ID");

        var report = await _reportService.GenerateFraudReportAsync(dto.ClaimId);
        return Ok(report);
    }

    // GET: Reports by type
    [HttpGet("type/{type}")]
    public async Task<ActionResult> GetByType(string type)
    {
        var reports = await _claimsReportRepository.GetByTypeAsync(type);
        var dtos = _mapper.Map<IEnumerable<ClaimsReportDto>>(reports);
        return Ok(dtos);
    }

    // GET: Reports by date range
    [HttpGet("range")]
    public async Task<ActionResult> GetByDateRange(DateTime from, DateTime to)
    {

```

```

        var reports = await _claimsReportRepository.GetByDateRangeAsync(from, to);
        var dtos = _mapper.Map<IEnumerable<ClaimsReportDto>>(reports);
        return Ok(dtos);
    }

    [HttpGet("download/{reportId}")]
    public async Task<IActionResult> DownloadReport(int reportId)
    {
        var reportDto = await _reportService.GetByIdAsync(reportId);
        if (reportDto == null)
        {
            return NotFound($"Report with ID {reportId} not found.");
        }
        var pdfBytes = GeneratePdf(reportDto);
        return File(pdfBytes, "application/pdf", $"ClaimReport_{reportId}.pdf");
    }

    // Helper: Generate simple PDF content (placeholder)
    private byte[] GeneratePdf(ClaimsReportDto report)
    {
        using var ms = new MemoryStream();
        using var writer = new StreamWriter(ms, Encoding.UTF8, leaveOpen: true);
        writer.WriteLine("ClaimWise Report");
        writer.WriteLine("-----");
        writer.WriteLine($"Report ID: {report.ReportID}");
        writer.WriteLine($"Claim ID: {report.ClaimID}");
        writer.WriteLine($"Policyholder ID: {report.PolicyholderID}");
        writer.WriteLine($"Type: {report.Type}");
        writer.WriteLine($"Generated Date: {report.GeneratedDate}");
        writer.WriteLine($"Insights: {report.Insights}");
        writer.Flush();
    }

```

```
        return ms.ToArray(); // Replace with real PDF generation later
    }
}
}
```

Frontend (Angular + Bootstrap)

The frontend provides an interactive interface for users to generate, view, and analyze claim reports. It enables users to select filters, request report generation, and view the output in a readable format.

Components

- **Reports:** This component handles the display and management of generated reports. It allows users to apply filters (like claim date, status, or hospital) and view summarized results.
-

Services

- **Reports Service:** Communicates with backend APIs to send report generation requests and fetch the generated report data for display.

CODE (Report Service)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
export interface ClaimsReportDto {
    reportID: number;
    claimID: number;
    policyholderID: number;
    type: string;
    generatedDate: string;
    insights: string;
```

```

}

export interface GenerateReportRequestDto {
    claimId: number;
}

@Injectables({ providedIn: 'root' })
export class ReportsService {
    private baseUrl = 'http://localhost:5030/api/claimsreport';
    constructor(private http: HttpClient) {}

    // Get all reports
    getAllReports(): Observable<ClaimsReportDto[]> {
        return this.http.get<ClaimsReportDto[]>(this.baseUrl);
    }

    // Generate report for a claim
    generateReport(dto: GenerateReportRequestDto): Observable<string> {
        return this.http.post(this.baseUrl + '/Auto-generateReport', dto, {
            responseType: 'text'
        });
    }

    // Filter by type
    getReportsByType(type: string): Observable<ClaimsReportDto[]> {
        return this.http.get<ClaimsReportDto[]>(`${this.baseUrl}/type/${type}`);
    }

    // Filter by date range
    getReportsByDateRange(from: string, to: string): Observable<ClaimsReportDto[]> {
        return this.http.get<ClaimsReportDto[]>(`${this.baseUrl}/range?from=${from}&to=${to}`);
    }
}

```

```
// Get report by ID
```

```
getReportById(reportId: number): Observable<ClaimsReportDto> {  
    return this.http.get<ClaimsReportDto>(`${this.baseUrl}/${reportId}`);  
}
```

```
// Download report as PDF
```

```
downloadReportById(reportId: number): Observable<Blob> {  
    return this.http.get(`${this.baseUrl}/download/${reportId}`, {  
        responseType: 'blob'  
    });  
}  
}
```